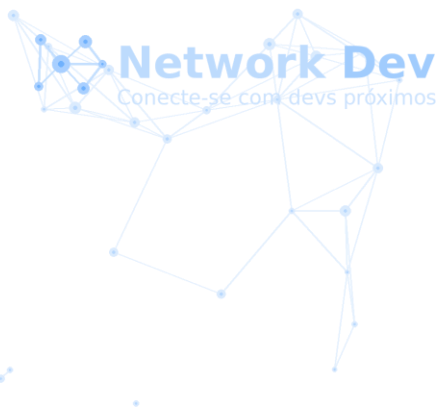




## **Centro Universitário UniFECAF**

Curso de Análise e Desenvolvimento de Sistemas

Adriel Pereira de Almeida Santos — RA: 78944

Emilly de Souza — RA: 104584

Joice Barbosa Santos — RA: 102859

Luiz Henrique de Almeida Santos — RA: 88078

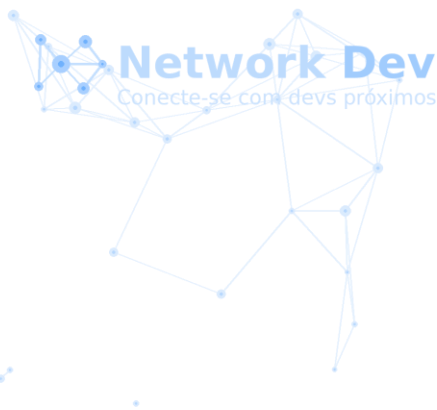
Matheus Bondenzan de Souza — RA: 96122

### **NETWORK DEV**

### **APLICATIVO MOBILE DE NETWORKING ENTRE DESENVOLVEDORES VIA BLUETOOTH LOW ENERGY (BLE)**

Taboão da Serra — SP

2026



Adriel Pereira de Almeida Santos — RA: 78944

Emilly de Souza — RA: 104584

Joice Barbosa Santos — RA: 102859

Luiz Henrique de Almeida Santos — RA: 88078

Matheus Bondenzan de Souza — RA: 96122

**NETWORK DEV**  
**APLICATIVO MOBILE DE NETWORKING ENTRE**  
**DESENVOLVEDORES VIA BLUETOOTH LOW ENERGY (BLE)**

Trabalho de conclusão de semestre apresentado  
ao curso de Análise e Desenvolvimento de  
Sistemas da UniFECAF, como requisito para  
participação na ExpoTech 2026.

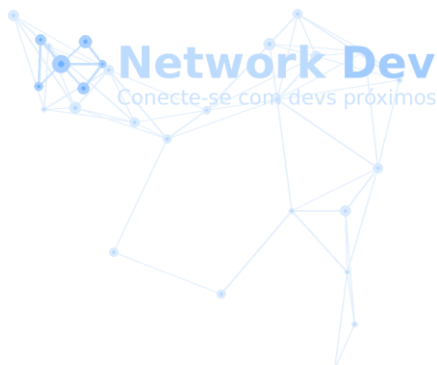
Orientador: Guilherme Henrique de Camargo e Silva

**Repositório Front-end:** <https://github.com/JoiceBsantos/network-dev>

**Repositório Back-end:** <https://github.com/JoiceBsantos/network-dev-back>

Taboão da Serra — SP

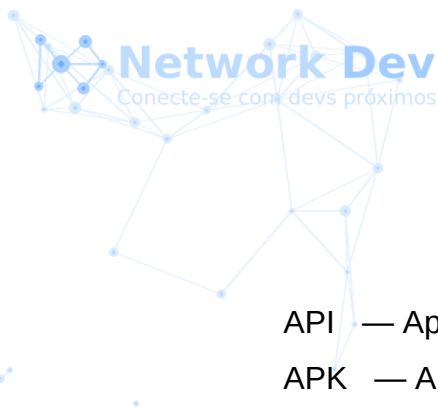
2026



## RESUMO

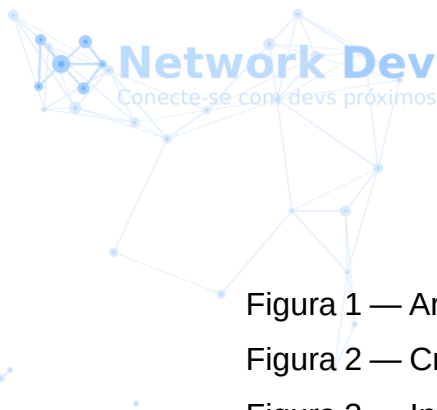
O Network Dev é um aplicativo mobile híbrido desenvolvido em React Native com TypeScript, voltado ao networking entre desenvolvedores de tecnologia. A solução utiliza sensores ESP32 via Bluetooth Low Energy (BLE) para detectar dispositivos próximos e exibir perfis de desenvolvedores em tempo real, permitindo o envio e gerenciamento de solicitações de conexão profissional. O back-end foi desenvolvido em Spring Boot 3.x, com persistência de dados em MySQL para dados relacionais e MongoDB para logs de proximidade BLE. As imagens de perfil e publicações são armazenadas no Firebase Storage. O aplicativo conta com feed de publicações com suporte a curtidas e upload de imagens, sistema de conexões com notificações de status, perfil editável com stacks tecnológicas, e interface responsiva compatível com Android, iOS e Web. O monitoramento da API é realizado em tempo real por meio de Prometheus e Grafana, garantindo observabilidade e qualidade do serviço.

Palavras-chave: React Native. Bluetooth Low Energy. ESP32. Spring Boot. Networking. IoT. Firebase.



## LISTA DE ABREVIATURAS E SIGLAS

- API — Application Programming Interface
- APK — Android Package Kit
- ADS — Análise e Desenvolvimento de Sistemas
- BLE — Bluetooth Low Energy
- ESP32 — Microcontrolador com suporte a Bluetooth e Wi-Fi
- IoT — Internet of Things (Internet das Coisas)
- JVM — Java Virtual Machine
- MySQL — My Structured Query Language
- QA — Quality Assurance
- REST — Representational State Transfer
- UI — User Interface (Interface do Usuário)
- UX — User Experience (Experiência do Usuário)



## LISTA DE FIGURAS

|                                                                                                          |    |
|----------------------------------------------------------------------------------------------------------|----|
| Figura 1 — Arquitetura do Sistema Network Dev .....                                                      | 12 |
| Figura 2 — Criação do esqueleto do app .....                                                             | 14 |
| Figura 3 — Interfaces do aplicativo mobile Network Dev .....                                             | 15 |
| Figura 4 — MySQL Workbench — Tabelas do banco proxnet .....                                              | 16 |
| Figura 5 — MySQL Workbench — Script SQL para geração da estrutura física do banco de dados proxnet ..... | 17 |
| Figura 6 — MongoDB Compass — proxnet_logs .....                                                          | 17 |
| Figura 7 — Targets Prometheus com status UP .....                                                        | 18 |
| Figura 8 — Dashboard Grafana — JVM Micrometer .....                                                      | 19 |
| Figura 9 — Dashboard Grafana — Contagem de Requisições HTTP .....                                        | 19 |
| Figura 10 — Dashboard Grafana — Tempo de Resposta por Endpoint.....                                      | 19 |

## LISTA DE QUADROS

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Quadro 1 — Especificações técnicas dos componentes de hardware.....      | 10 |
| Quadro 2 — Disciplinas aplicadas no desenvolvimento do Network Dev ..... | 11 |



## SUMÁRIO

|                                            |    |
|--------------------------------------------|----|
| 1 INTRODUÇÃO .....                         | 7  |
| 1.1 Contexto e Motivação .....             | 7  |
| 1.2 Objetivos .....                        | 7  |
| 1.3 Justificativa .....                    | 7  |
| 2 TECNOLOGIAS UTILIZADAS .....             | 8  |
| 2.1 Front-end .....                        | 8  |
| 2.2 Back-end .....                         | 9  |
| 2.3 Banco de Dados .....                   | 9  |
| 2.4 IoT — ESP32 e BLE .....                | 9  |
| 2.5 Firebase Storage .....                 | 10 |
| 2.6 Prometheus e Grafana .....             | 10 |
| 2.7 Disciplinas Aplicadas no Projeto ..... | 11 |
| 3 ARQUITETURA DO SISTEMA .....             | 12 |
| 3.1 Visão Geral .....                      | 12 |
| 4 FUNCIONALIDADES DO APLICATIVO .....      | 13 |
| 4.1 Autenticação .....                     | 13 |
| 4.2 Home — Radar BLE .....                 | 13 |
| 4.3 Detecção e Conexão entre Devs .....    | 13 |
| 4.4 Feed de Publicações .....              | 13 |
| 4.5 Sistema de Conexões .....              | 13 |
| 4.6 Perfil do Usuário .....                | 13 |
| 5 TELAS DO APLICATIVO .....                | 14 |
| 6 BANCO DE DADOS .....                     | 16 |
| 6.1 Modelo Relacional — MySQL .....        | 16 |
| 6.2 Banco NoSQL — MongoDB .....            | 17 |
| 7 QUALITY ASSURANCE E MONITORAMENTO .....  | 18 |
| 7.1 Monitoramento com Prometheus .....     | 18 |
| 7.2 Dashboards Grafana .....               | 18 |
| 8 CONSIDERAÇÕES FINAIS .....               | 20 |
| REFERÊNCIAS .....                          | 21 |

# 1 INTRODUÇÃO

## 1.1 Contexto e Motivação

O mercado de tecnologia é caracterizado pela alta competitividade e pela constante necessidade de atualização profissional. Nesse cenário, o networking entre desenvolvedores torna-se uma ferramenta estratégica para a troca de conhecimento, formação de equipes e identificação de oportunidades de trabalho. Ferramentas tradicionais de networking como LinkedIn apresentam limitações: são assíncronas e dependem de busca ativa. O Network Dev surge como alternativa inovadora ao propor conexões profissionais em tempo real, baseadas em proximidade física detectada por Bluetooth Low Energy (BLE) através de sensores ESP32.

## 1.2 Objetivos

O objetivo geral é desenvolver um aplicativo mobile híbrido que permita a descoberta e o networking entre desenvolvedores em tempo real, utilizando sensores IoT para detecção de proximidade via BLE.

Os objetivos específicos são:

- Desenvolver interface mobile responsiva em React Native compatível com Android, iOS e Web;
- Implementar integração com dispositivos ESP32 via BLE;
- Criar API REST em Spring Boot para gerenciamento de usuários, conexões e publicações;
- Integrar banco de dados relacional MySQL e NoSQL MongoDB;
- Implementar armazenamento de mídias no Firebase Storage;
- Configurar monitoramento da API com Prometheus e Grafana.

## 1.3 Justificativa

O projeto justifica-se pela crescente demanda por ferramentas de networking no setor de tecnologia e pela oportunidade de aplicar conceitos interdisciplinares do curso de ADS, incluindo desenvolvimento mobile, back-end, banco de dados, IoT e monitoramento de sistemas.

## 2 TECNOLOGIAS UTILIZADAS

### 2.1 Front-end

O front-end foi desenvolvido com React Native, framework da Meta para criação de aplicativos móveis híbridos utilizando TypeScript. Bibliotecas utilizadas: Expo SDK, React Navigation v7 (Stack + Bottom Tabs), Axios, AsyncStorage, Expo Image Picker, Lottie React Native, Expo Linear Gradient e TypeScript.

Repositório: <https://github.com/JoiceBsantos/network-dev>

#### Estrutura do projeto:

```
network-dev/  
├── src/  
│   ├── assets/           # Imagens e animações  
│   │   └── stacks/       # Ícones das tecnologias  
│   ├── navigation/  
│   │   └── AppNavigator.tsx  
│   ├── screens/  
│   │   ├── LoginScreen.tsx  
│   │   ├── CreateProfileScreen.tsx  
│   │   ├── HomeScreen.tsx  
│   │   ├── FeedScreen.tsx  
│   │   ├── ConnectionScreen.tsx  
│   │   ├── ConnectionsScreen.tsx  
│   │   └── MyProfileScreen.tsx  
│   ├── services/  
│   │   ├── api.ts  
│   │   ├── auth.ts  
│   │   └── userService.ts  
│   ├── config/  
│   │   └── firebase.ts  
│   └── utils/  
│       └── responsive.ts  
├── App.tsx  
├── app.json  
└── package.json
```



## 2.2 Back-end

O back-end foi desenvolvido em Spring Boot 3.x com Java 17. Componentes: Spring Data JPA, Spring Data MongoDB, Spring Security, Spring Boot Actuator, Micrometer Prometheus, Firebase Admin SDK e SpringDoc OpenAPI (Swagger UI).

Repositório: <https://github.com/JoiceBsantos/network-dev-back>

### Estrutura do projeto:

```
network-dev-back/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/proxnet/
│   │   │       ├── controller/
│   │   │       ├── service/
│   │   │       ├── repository/
│   │   │       ├── model/
│   │   │       └── config/
│   │   └── resources/
│   │       ├── application.yml
│   │       └── serviceAccountKey.json
├── NET-DEV/          # Firmware ESP32
├── create_database.sql
├── pom.xml
└── README.md
```

## 2.3 Banco de Dados

O MySQL 8.0 armazena dados relacionais: usuários, posts, curtidas e conexões. O MongoDB armazena logs de proximidade BLE gerados pelos sensores ESP32, dados não estruturados gerados em alta frequência.

## 2.4 IoT — ESP32 e BLE

O hardware do projeto é composto pelo microcontrolador ESP32 e pelo módulo RTC DS3231. O ESP32 transmite o UUID do desenvolvedor via Bluetooth Low Energy (BLE), permitindo que o aplicativo detecte dispositivos próximos e exiba o perfil correspondente. Os dados de proximidade (RSSI) são registrados no MongoDB via

endpoint `/api/proximity/detect`. O RTC DS3231 sincroniza o timestamp com o servidor via `/api/rtc/sync`, garantindo a precisão temporal dos logs.

| Especificação        | ESP32 DevKit V1                | Módulo RTC DS3231          | Protoboard 830pt         |
|----------------------|--------------------------------|----------------------------|--------------------------|
| Imagem               |                                |                            |                          |
| Fabricante / Chip    | Espressif / ESP32-WROOM-32     | Maxim Integrated / DS3231  | Genérico                 |
| Função               | Microcontrolador IoT           | Relógio Tempo Real (RTC)   | Prototipagem de circuito |
| CPU / Frequência     | Dual-core Xtensa LX6 / 240 MHz | N/A                        | N/A                      |
| Wi-Fi / Bluetooth    | 802.11 b/g/n 2.4 GHz + BLE 4.2 | N/A                        | N/A                      |
| Memória              | 520 KB RAM / 4 MB Flash        | N/A                        | N/A                      |
| Interface            | Micro USB / GPIO (30 pinos)    | I2C (SDA / SCL)            | 830 pontos de conexão    |
| Tensão operação      | 3.3V (entrada USB 5V)          | 3.3V — 5.5V                | N/A                      |
| Precisão / Pontos    | N/A                            | ±2 ppm (alta precisão)     | 830 pontos               |
| Dimensões            | 23,4 × 51,5 mm                 | 38 × 22 mm                 | 165 × 55 mm              |
| Aplicação no projeto | Processamento BLE e Wi-Fi      | Marcação de tempo dos logs | Conexão dos componentes  |

Quadro 1 — Especificações técnicas dos componentes de hardware do protótipo. Fonte: Os autores (2026).

### 2.5 Firebase Storage

O Firebase Storage é utilizado para armazenamento de imagens de perfil e publicações do feed. A integração é realizada pelo back-end Spring Boot através do Firebase Admin SDK, garantindo que as URLs públicas das imagens sejam armazenadas no MySQL e acessadas diretamente pelo front-end.

### 2.6 Prometheus e Grafana

O monitoramento da API é realizado através da stack Prometheus e Grafana. O Spring Boot Actuator expõe métricas no endpoint `/actuator/prometheus`, coletadas pelo Prometheus a cada 15 segundos. O Grafana visualiza as métricas em dashboards interativos em tempo real. Os dashboards configurados são: JVM Micrometer (ID 4701) e Spring Boot 3.x Statistics (ID 19004).

### 2.7 Disciplinas Aplicadas no Projeto

O desenvolvimento do Network Dev integrou conhecimentos de diversas disciplinas do curso de Análise e Desenvolvimento de Sistemas da UniFECAF, conforme apresentado no Quadro 2.

| Disciplina                                   | Aplicação no Projeto                               |
|----------------------------------------------|----------------------------------------------------|
| Mobile Development                           | Desenvolvimento em React Native, Expo e TypeScript |
| Quality Assurance                            | Monitoramento com Prometheus e Grafana             |
| Disruptive Architectures: IoT, Big Data e IA | Integração com ESP32 via BLE e MongoDB             |
| Web Programming for Back End                 | API REST desenvolvida em Spring Boot               |
| Web Programming for Front End                | Interface responsiva em React Native               |
| Database Modeling & SQL                      | Modelagem e implementação do banco MySQL           |
| Software Engineering                         | Arquitetura cliente-servidor e padrões REST        |
| UX/UI Design & Design Thinking               | Design das telas e experiência do usuário          |
| Object Oriented Programming                  | Desenvolvimento orientado a objetos em Java        |

Quadro 2 — Disciplinas aplicadas no desenvolvimento do Network Dev. Fonte: Os autores (2026).

## 3 ARQUITETURA DO SISTEMA

### 3.1 Visão Geral

A arquitetura do Network Dev segue o padrão cliente-servidor, composta por quatro camadas principais: dispositivo IoT (ESP32), aplicativo mobile (React Native), API REST (Spring Boot) e camada de dados (MySQL + MongoDB + Firebase Storage).

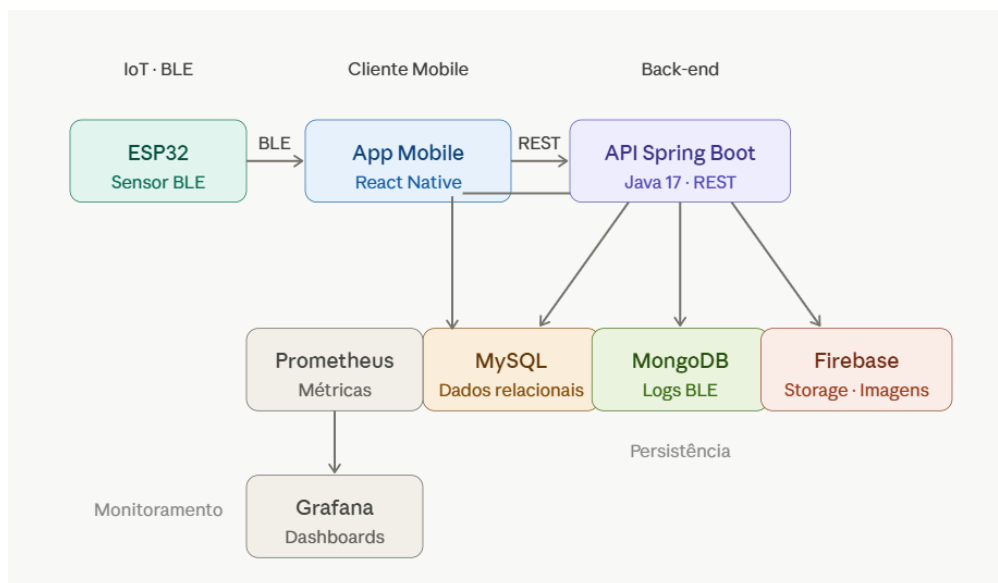


Figura 1 — Arquitetura do Sistema Network Dev. Fonte: Os autores (2026).

O ESP32 transmite o UUID do desenvolvedor via BLE, que é captado pelo aplicativo mobile. O app consulta a API REST para obter o perfil detectado e registra o log de proximidade no MongoDB. O usuário então envia uma solicitação de conexão, armazenada no MySQL, e o receptor recebe a notificação podendo aceitá-la ou recusá-la. Todo o processo é monitorado em tempo real pelo Prometheus e Grafana.

## **4 FUNCIONALIDADES DO APLICATIVO**

### **4.1 Autenticação**

Login com email e senha usando hash BCrypt, com sessão persistida via AsyncStorage e logout que limpa os dados locais. A autenticação não utiliza JWT — o userId é salvo localmente via AsyncStorage após o login validado pelo back-end com hash BCrypt.

### **4.2 Home — Radar BLE**

Radar animado que escaneia o ambiente BLE, exibindo cards de desenvolvedores detectados com nome, stack e compatibilidade. Usuários com solicitação já enviada recebem o status "Aguardando resposta".

### **4.3 Detecção e Conexão entre Devs**

Tela exibe fotos reais do Firebase, objetivo profissional e compatibilidade dos dois desenvolvedores. Ao confirmar, uma animação de handshake é exibida e a solicitação é enviada à API.

### **4.4 Feed de Publicações**

Feed com posts reais da API ordenados por data, suportando curtidas com animação, criação de posts com texto e imagem, visualização fullscreen e infinite scroll com recarregamento automático.

### **4.5 Sistema de Conexões**

Quatro seções: solicitações recebidas (aceitar/recusar), conexões aceitas, solicitações pendentes e notificações de recusadas em vermelho com opção de dispensar.

### **4.6 Perfil do Usuário**

Exibe foto, nome, cargo, stacks com ícones, contador de conexões e posts, bio e objetivo. Permite edição completa, upload de foto para Firebase e grid de publicações em 3 colunas. Permite também a exclusão de publicações com sincronização automática com a API e o Firebase Storage.

## 5 TELAS DO APLICATIVO

As telas a seguir apresentam os wireframes do aplicativo Network Dev, desenvolvidos para ilustrar a estrutura visual e o fluxo de navegação entre as principais funcionalidades do sistema.



Figura 2 — Criação do esqueleto do app. Fonte: Os autores (2026).

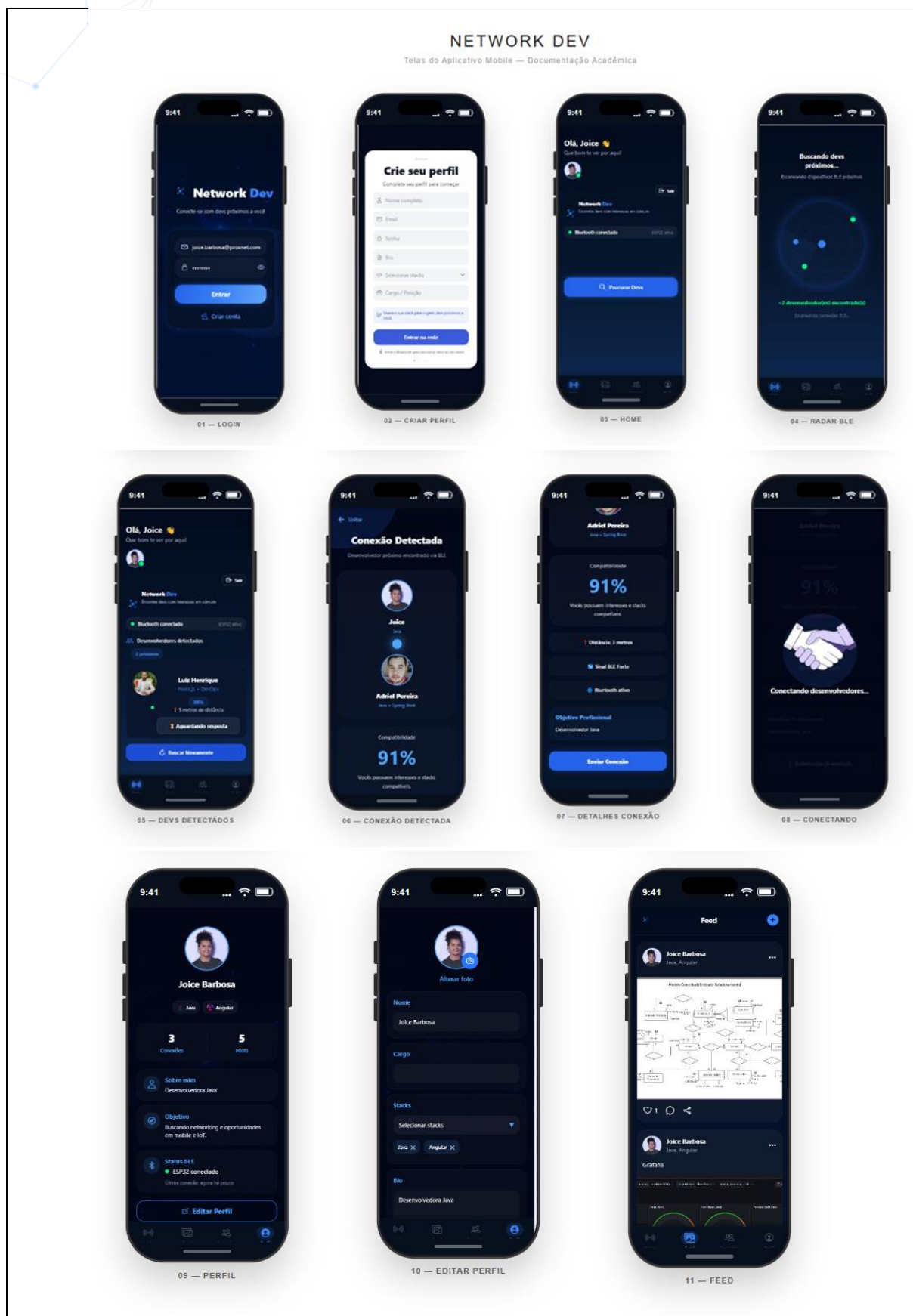


Figura 3 — Interfaces do aplicativo mobile Network Dev: (Login, Criar Perfil, Home, Radar BLE, Devs Detectados, Conexão Detectada, Detalhes da Conexão, Conectando, Perfil, Editar Perfil e Feed). Fonte: Os autores (2026).

O banco de dados relacional proxnet possui quatro tabelas principais: users, que armazena os dados dos desenvolvedores (id, name, email, password\_hash, bio, stack, position, uuid\_bluetooth, profile\_image\_url); posts, que registra as publicações do feed (id, user\_id, description, image\_url, created\_at); post\_likes, que controla as curtidas (id, post\_id, user\_id, created\_at); e connections, que gerencia as solicitações e conexões entre desenvolvedores (id, requester\_id, receiver\_id, status, created\_at), onde o campo status pode assumir os valores PENDING, ACCEPTED ou DECLINED.

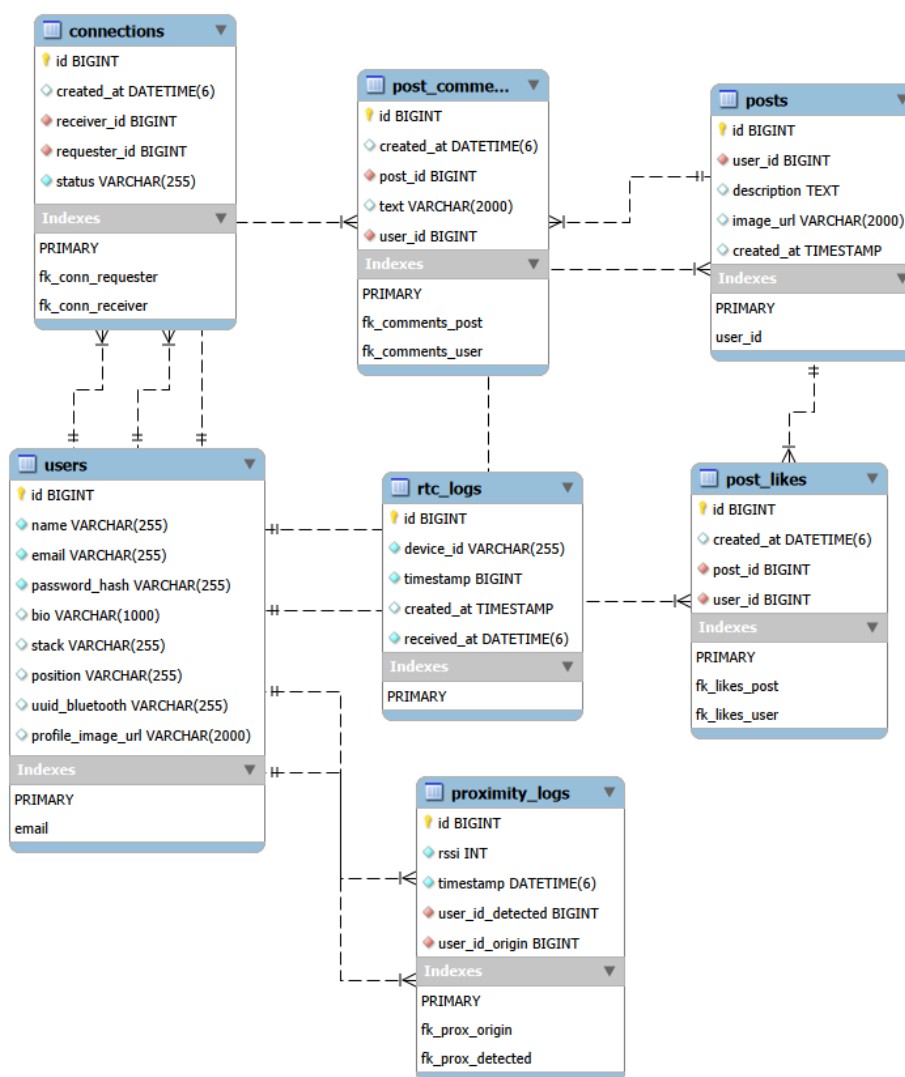


Figura 4 — MySQL Workbench — Tabelas do banco proxnet. Fonte: Os autores (2026).



```
sql

USE proxnet;

SELECT
  CONCAT(
    'CREATE TABLE ', TABLE_NAME, ' (\n',
    GROUP_CONCAT(
      CONCAT(' ', COLUMN_NAME, ' ', COLUMN_TYPE,
        IF(IS_NULLABLE='NO', ' NOT NULL', ''),
        IF(COLUMN_KEY='PRI', ' PRIMARY KEY', ''))
      ) SEPARATOR ',\n'
    ),
    '\n);'
  ) AS codigo
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'proxnet'
GROUP BY TABLE_NAME;
```

Figura 5 — MySQL Workbench — Script SQL para geração da estrutura física do banco de dados proxnet. Fonte: Os autores (2026).

## 6.2 Banco NoSQL — MongoDB

O MongoDB armazena os logs de proximidade BLE no banco proxnet\_logs. Cada documento representa um evento de detecção entre dois dispositivos, contendo os campos originUserId, detectedUserId, rssi (intensidade do sinal Bluetooth) e timestamp.

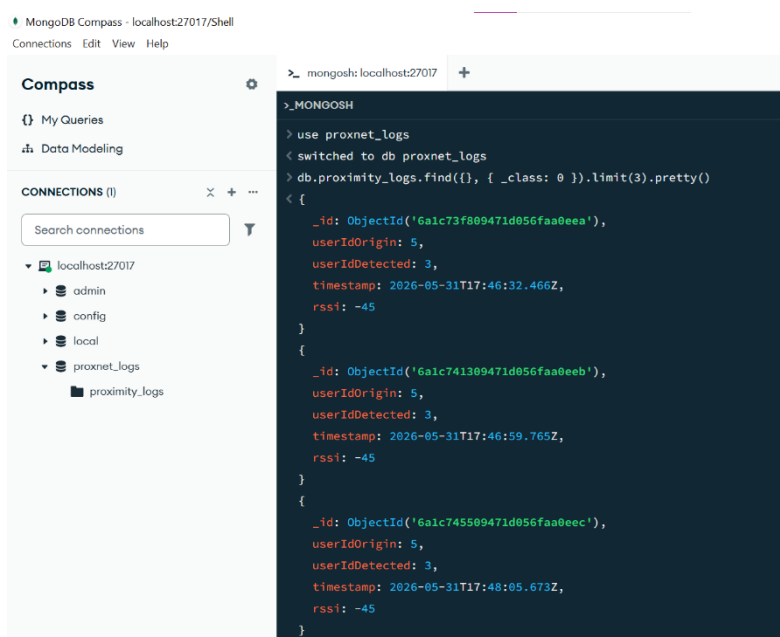


Figura 6 — MongoDB Compass — proxnet\_logs. Fonte: Os autores (2026).

## 7 QUALITY ASSURANCE E MONITORAMENTO

### 7.1 Monitoramento com Prometheus

O Spring Boot Actuator foi configurado para expor métricas no endpoint `/actuator/prometheus`, coletadas pelo Prometheus a cada 15 segundos. Os targets configurados são o próprio Prometheus (`localhost:9090`) e a API do Network Dev (`localhost:8080`), ambos com status UP, confirmando o funcionamento correto do monitoramento.

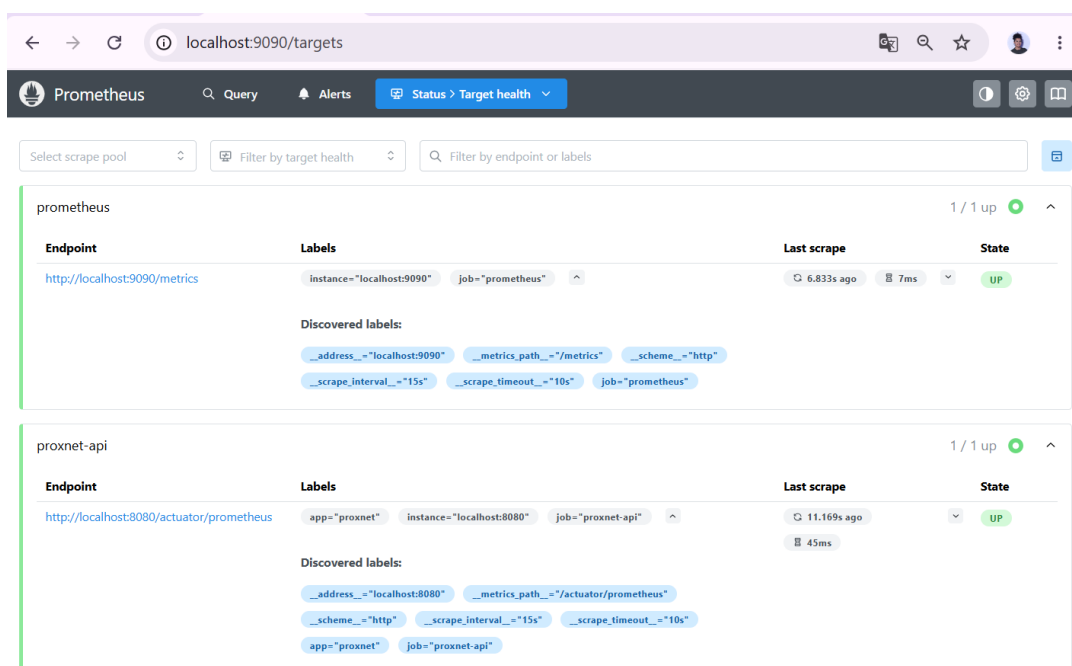


Figura 7 — Targets Prometheus com status UP. Fonte: Os autores (2026).

### 7.2 Dashboards Grafana

Foram configurados dois dashboards no Grafana para monitoramento em tempo real. O dashboard JVM Micrometer (ID 4701) monitora métricas da JVM incluindo uso de heap, threads ativos, Garbage Collection e CPU usage. O dashboard Spring Boot 3.x Statistics (ID 19004) monitora uptime, heap e non-heap, conexões HikariCP ao banco MySQL, e estatísticas HTTP com tempo de resposta por endpoint. Dados reais registraram tempo médio de 42,8ms para o endpoint `/api/posts` e 17,6ms para `/api/users/{id}`.



Figura 8 — Dashboard Grafana — JVM Micrometer. Fonte: Os autores (2026).

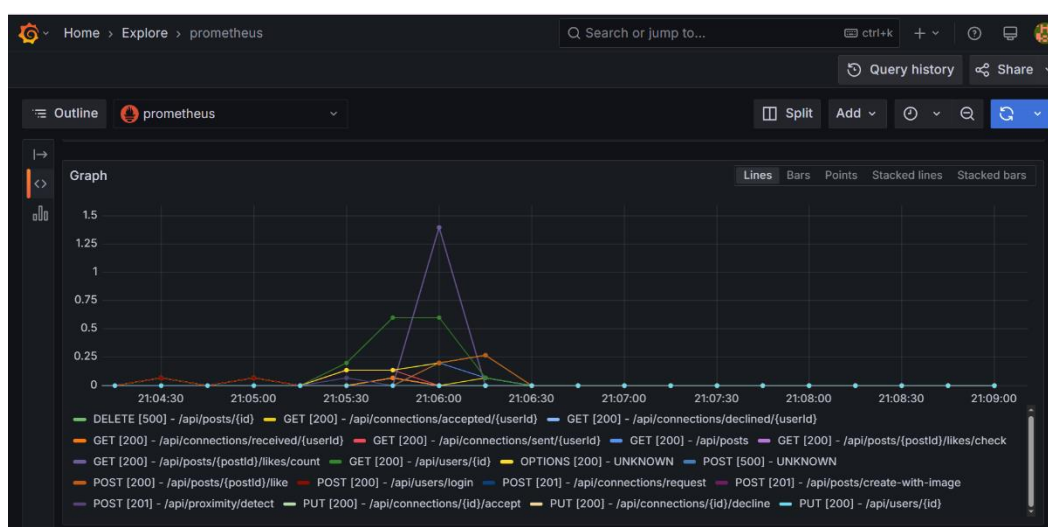


Figura 9 — Dashboard Grafana — Contagem de Requisições HTTP. Fonte: Os autores (2026).

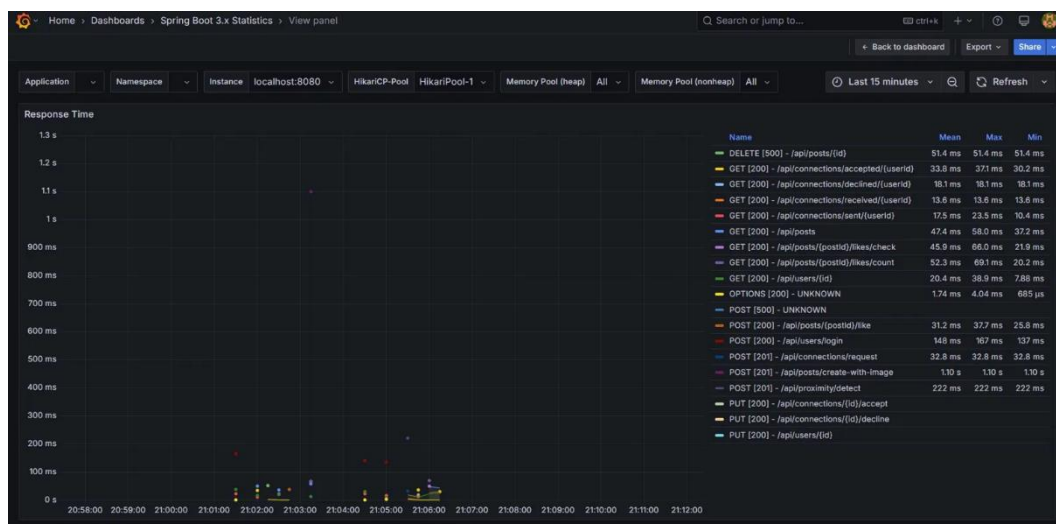


Figura 10 — Dashboard Grafana — Tempo de Resposta por Endpoint. Fonte: Os autores (2026).

## 8 CONSIDERAÇÕES FINAIS

O Network Dev foi desenvolvido com sucesso, atendendo aos requisitos estabelecidos para a trilha Mobile Development da ExpoTech 2026. A integração entre React Native, Spring Boot, MySQL, MongoDB, Firebase e ESP32 demonstrou a viabilidade técnica de uma solução de networking baseada em proximidade física via Bluetooth Low Energy.

Os principais desafios enfrentados durante o desenvolvimento foram a integração entre plataformas distintas para upload de imagens na web e no mobile, a configuração do Firebase Storage com Spring Boot, e o desenvolvimento do sistema de conexões com notificações de status em tempo real.

O monitoramento da API com Prometheus e Grafana comprovou a estabilidade do sistema, registrando tempo médio de resposta de 42,8ms para o endpoint de posts e 17,6ms para o endpoint de usuários, valores dentro dos padrões aceitáveis para aplicações REST.

Como trabalhos futuros, propõe-se a integração real com o firmware ESP32 para escaneamento BLE efetivo, implementação de notificações push, sistema de chat entre conexões aceitas, e deploy em nuvem com pipeline CI/CD automatizado via GitHub Actions.

## REFERÊNCIAS

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 7. ed. São Paulo: Pearson, 2019.

DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson, 2017.

FOWLER, Martin. Padrões de Arquitetura de Aplicações Corporativas. Porto Alegre: Bookman, 2006.

NIELSEN, Jakob. Usabilidade na Web. Rio de Janeiro: Elsevier, 2007.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

META PLATFORMS INC. React Native Documentation. Disponível em: <https://reactnative.dev>. Acesso em: maio 2026.

PIVOTAL SOFTWARE. Spring Boot Reference Documentation. Disponível em: <https://docs.spring.io/spring-boot>. Acesso em: maio 2026.

GOOGLE LLC. Firebase Documentation. Disponível em: <https://firebase.google.com/docs>. Acesso em: maio 2026.

PROMETHEUS AUTHORS. Prometheus Documentation. Disponível em: <https://prometheus.io/docs>. Acesso em: maio 2026.

GRAFANA LABS. Grafana Documentation. Disponível em: <https://grafana.com/docs>. Acesso em: maio 2026.

ESPRESSIF SYSTEMS. ESP32 Technical Reference Manual. Disponível em: <https://www.espressif.com>. Acesso em: maio 2026.

ORACLE CORPORATION. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc>. Acesso em: maio 2026.

MONGODB INC. MongoDB Manual. Disponível em: <https://www.mongodb.com/docs>. Acesso em: jun 2026.

EXPO DEV. Expo Documentation. Disponível em: <https://docs.expo.dev>. Acesso em: maio 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.